

알고리즘 보고서 작성

로봇 21기 예비단원 최윤서

1. 알고리즘이란?

- 어떤 목적을 달성하거나 결과물을 만들어내기 위해 거쳐야하는 일련의 과정들의 집합을 의미
- 가는 루트는 다양하며 여러가지 상황에 따른 알고리즘은 모두 다름
- 따라서, 시간복잡도가 가장 낮은 알고리즘을 선택하여 사용

2. 복잡도

2-1. 복잡도란?

- 알고리즘의 성능을 나타내는 척도
- 시간복잡도와 공간복잡도가 존재함

2-2. 시간복잡도

- 특정한 크기의 입력 -> 연산이 몇번 이루어지는지 표기
- 간단히 말해 '시간 효율성'

2-3. 공간복잡도

- 특정한 크기의 입력 -> 얼마나 많은 메모리를 차지하는지를 의미
- 간단히 말해 '메모리 효율성'

3, 빅오(Big O) 표기법

3-1. 빅오표기법이란?

- 가장 빠르게 증가하는 항만 고려하는 표기법
- 알고리즘의 효율성을 표기해주는 표기법

3-2. 왜 빅오 표기법을 사용하는가?

- 알고리즘 효율성을 상한선 기준으로 표현하기 때문
-> 알고리즘 효율성은 값이 클수록, 그래프가 위로 향할수록 비효율적
- 근데 빅오메가 표기법은 하한선을 기준으로하고
빅세타 표기법은 상한선과 하한선을 기준으로 표현함

3-3. 빅오 표기법의 수학적 정의

- 모든 $n \geq n_0 \geq 0$ 에 대하여 $0 \leq f(n) \leq kg(n)$ 인 양의 상수 k 와 n_0 가 존재하면 $f(n) = O(g(n))$ 이다.
- 예시)
 $f(n) = n^2 + 2n + 3 / g(n) = n^2$ 라고 가정하자
 $n^2 + 2n + 3 \leq kn^2$ 이 성립하는 k 가 존재하기 때문에 $f(n) = O(n^2)$ 으로 나타낼 수 있다.

3-4. 특징

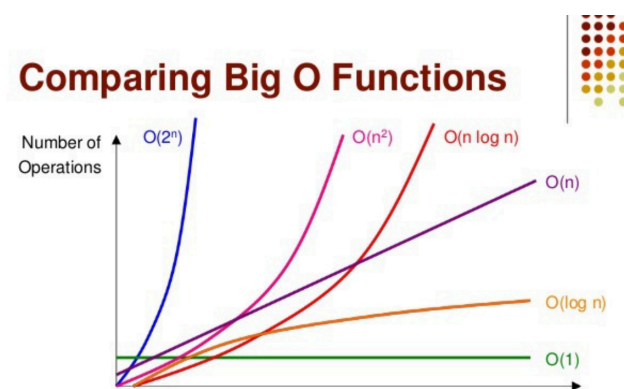
3-4-1. 상수항 무시

- 빅오 표기법은 데이터 입력값이 충분히 크다고 가정하고있음
- 알고리즘의 효율성 또한 데이터 입력값의 크기에 영향 받음
- 위 두가지 이유 때문에 상수항 같은 사소한 부분은 무시함.

3-4-2. 영향력 없는 항 무시

- 입력값의 크기에 따라 영향 받음
- 따라서 영향력이 큰 항 이외의 항들은 무시
- $O(n^2 + 2n + 1) \rightarrow O(n^2)$ 와 같이 n^2 이외의 항은 무시

3-5. 빅오 표기법 성능 비교



- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- 왼쪽에서 오른쪽으로 갈수록 효율성 떨어짐

4. 빅세타 표기법

4-1. 빅세타 표기법이란?

- 빅 오와 빅 오메가 표기법의 공통부분
- 알고리즘 실행 시간이 상한선과 하한선의 동일한 증가율을 가질 때 이를 정확하게 감싸서 표현하는 점근적 표기법
- 알고리즘의 성능이 측정 함수와 위아래로 같은 비율로 성장함을 의미

4-2. 수학적 의미

두 함수 $f(n)$, $g(n)$ 이 있을 때, $n_1 \leq n$, $C_1 * g(n) \leq f(n) \leq C_2 * g(n)$ 이 성립하는 상수 C_1 , C_2 , n_1 이 존재하면 $f(n) = \theta(g(n))$ 이다.

5. 빅오메가 표기법

5-1. 빅오메가 표기법이란?

- 빅 오 표기법과 반대되는 개념
- 알고리즘의 실행 시간이 최소한 어느 정도 걸리는지 나타내는 점근적 하한선 표기법
- 점근적 하한: 입력 크기 n 이 커질 때, 알고리즘의 실행 시간이 밑에서부터 제한하는 최소한의 증가율

5-2. 수학적 의미

두 함수 $f(n)$, $g(n)$ 이 있을 때, $n_1 \leq n$, $C_1 * g(n) \leq f(n) \leq C_2 * g(n)$ 이 성립하는 상수 C_1 , C_2 , n_1 이 존재하면 $f(n) = \theta(g(n))$ 이다.

6. A* 알고리즘

6-1. 개념

6-1-1. A* 알고리즘이란?

- 기존 다익스트라 알고리즘에 휴리스틱 개념을 더한 알고리즘
- 목적지 방향을 힌트로 삼아 불필요한 탐색을 줄이고 최단 경로 찾을
- 주로 내비게이션, 로봇 자율주행 경로 계획에 쓰임

6-2. A* 로직

6-2-1. Cost 공식: $f(n) = g(n) + h(n)$

- $g(n)$
 - 유클리디안 거리 사용 -> 두 점 사이 거리 공식과 동일함(피타고라스 정리) -> 대각선 이동 가능
 - 출발 노드에서 현재 노드까지 이동해야할 때 드는 최단 비용
-> 실제로 이동한 값만 축적됨
 - 상하좌우 한칸 당 10 cost, 대각선 한칸 당 14 cost 할당

-> 원래 대각선은 루트20이라서 14.1414지만 소수점 계산을 포함하면 속도가 느려질 수 있기 때문에 그냥 정수로 계산함
- $h(n)$
 - 현재 노드에서 목표 노드까지의 예상 이동 비용(전체 경로를 모두 이동했을 때 드는 cost 계산)
 - 맨하탄 거리 사용 -> 상하좌우만 이동 가능
 - 계속 갱신됨
-> 움직인 위치에서 목표 노트까지 실시간으로 계속 변화하기 때문
- $f(n)$
 - $g(n)$ 과 $h(n)$ 을 더한 값
 - 부모 노드에서 인접한 노드들 중에 $f(n)$ 값이 가장 작은 노드로 이동함
-> $f(n)$ 이 최소가 되는 방향으로 계속 이동

6-2-2. 알고리즘 작동 과정

- 1) 출발 노드를 open list에 넣어준다.
- 2) 출발 노드에 인접한 사각형 모두 open list에 넣어준다.
-> 인접한 사각형의 부모 노드 = 출발 노드
-> why? 경로 탐색 후 경로 생성해야할 때 사용
- 3) 인접한 노드 중에 $f(n)$ 이 가장 작은 노드 선택
- 4) 출발 노드를 open list -> close list로 위치 이동
-> close list에 들어가면 이미 지나간 길이라는 의미이고 이동한 노드의 부모 노드가 됨

5) 3)에서 선택한 노드에서 인접한 노드들을 모두 open list에 넣음
-> IF open list에 있는 값이 중복된다? (한칸만 이동했기 때문에 한두개 노드 정도는 겹칠 수 있음)

-> 기존에 open list에 있던 값보다 새로 들어온 노드의 $f(n)$ 이 큰지 작은지 판단
-> 기존 < new: 아무것도 실행X / 기존 > new: 그 중복되는 노드의 부모 노드를 현재 사각형으로 변경, 해당 사각형의 인접 사각형 $h(n)$, $g(n)$ 다시 계산하기

****왜 중복되는 노드를 계산해야하는가?**

- 한 방향으로만 진행된다면 계산할 필요가 없음(중첩돼도 그 방향으로 쭉 가면 됨)
- 그러나 앞에 장애물이 있는 경우도 있기 때문에 뒤로 돌아가야할 경우가 생길 수 있음(보통 $h(n)$ 을 계산하기 때문에 많이 발생하진 않음)
- 따라서 중첩됐을 때 한번쯤 검증해주는 단계가 필요함

6) 5)과정 작동 후 open list에 있는 노드들 중 $f(n)$ 이 최소가 되는 노드 선택 후 이동

7) 목표 노드가 open list에 들어오거나 open list가 빈 상태가 될 때까지 1)~6) 과정 반복

6-3. C++ 코드 구현

1) 우선순위 큐에 넣은 PQNode 구조체를 만들어준다

```
struct PQNode
{
    PQNode(int32 f, int32 g, Pos pos) : f(f), g(g), pos(pos) { }

    bool operator<(const PQNode& other) const { return f < other.f; }
    bool operator>(const PQNode& other) const { return f > other.f; }

    int32 f; // f = g + h
    int32 g;
    Pos pos;
};
```

2) 상하좌우, 대각선으로 움직일 때의 좌표 변화값과 이동했을 때의 비용을 정의

```
// F = G + H
// F = 최종 점수(작을 수록 좋음)
// G = 시작점에서 해당 좌표까지 이동하는데 드는 비용
// H = 목적지에서 해당 좌표까지 이동하는데 드는 비용

Pos start = _pos;
Pos dest = _board->GetExitPos(); // 목적지

Pos front[] =
{
    Pos(-1, 0), // UP
    Pos(0, -1), // LEFT
    Pos(1, 0), // DOWN
    Pos(0, 1), // RIGHT
    Pos(-1, -1), // UP_LEFT
    Pos(1, -1), // DOWN_LEFT
    Pos(1, 1), // DOWN_RIGHT
    Pos(-1, 1), // UP_RIGHT
};
```

```
int32 cost[] =
{
    10,
    10,
    10,
    10,
    14,
    14,
    14,
    14
};
```

- 상하좌우 비용을 1로 설정하면 대각선 비용이 1.41414.. 이기 때문에 계산하기 편하게 10으로 설정

3) 맵 사이즈 받아오기

```
const int32 size = _board->GetSize();
```

4) close list 생성 -> 이미 방문한 지역이라는 표식 남기기

```
// ClosedList -> closed[y][x] -> (y, x)에 방문을 했는지 여부
// Option) 사실 best만으로 판별 가능
vector<vector<bool>> closed(size, vector<bool>(size, false));
```

5) 노드의 부모를 추적할 수 있게 만들어줌

```
// 부모 추적 용도
vector<vector<Pos>> parent(size, vector<Pos>(size, Pos(-1, -1)))
```

6) Open list도 우선순위 큐로 만들어줌

```
// OpenList : 지금까지 발견된 목록
priority_queue<PQNode, vector<PQNode>, greater<PQNode>> pq;
```

7) 초기값 지정

```
// 초기값
{
    int32 g = 0;
    int32 h = 10 * (abs(dest.y - start.y) + abs(dest.x - start.x));

    pq.push(PQNode(g + h, g, start));
    best[start.y][start.x] = g + h;
    parent[start.y][start.x] = start;
}
```

- g는 시작점부터 현재 위치이므로 0 넣어줌
- h는 목적지에서 현재 위치 거리이기 때문에 목적지의 x,y - 현재 위치 x-y 빼주고 둘을 더해거 10 곱해주는 공식으로 만들

8) 우선순위 큐가 빌 때까지 순회하는 while문 작성

```
while (pq.empty() == false)
{
    // 제일 좋은 후보를 찾는다
    PQNode node = pq.top();
    pq.pop();

    // 동일한 좌표를 여러 경로로 찾아서
    // 더 빠른 경로로 인해서 이미 방문(closed)된 경우 스킵
    if (closed[node.pos.y][node.pos.x])
        continue;
    // 기껏 등록했더니만, 나보다 더 우수한 후보가 있다?
    if (best[node.pos.y][node.pos.x] < node.f)
        continue;

    // 방문
    closed[node.pos.y][node.pos.x] = true;

    // 목적지에 도착했으면 바로 종료
    if (node.pos == dest)
        break;
```

```
for (int32 dir = 0; dir < 8; dir++)
{
    Pos nextPos = node.pos + front[dir];
    // 갈 수 있는 지역은 있는지 확인
    if (CanGo(nextPos) == false)
        continue;
    if (closed[nextPos.y][nextPos.x])
        continue;

    int32 g = node.g + cost[dir];
    int32 h = 10 * (abs(dest.y - nextPos.y) + abs(dest.x - nextPos.x));

    // 다른 경로에서 더 빠른 길을 찾았으면 스킵
    if (best[nextPos.y][nextPos.x] <= g + h)
        continue;

    // 예약 진행
    best[nextPos.y][nextPos.x] = g + h;
    pq.push(PQNode(g + h, g, nextPos));
    parent[nextPos.y][nextPos.x] = node.pos;
}
```

- 제일 좋은 후보를 pq.top으로 뺏음 -> pop으로 내보내줌
- 이미 방문된 경우 스킵
- 방문한 후에는 closed를 true로 바꿔서 close list에 넣어줌

7. 다익스트라 알고리즘

7-1. 다익스트라 알고리즘이란?

- 그래프에서 하나의 시작 정점에서 다른 모든 정점까지의 최단 경로를 구하는 대표적인 최단 경로 탐색 알고리즘
- 간선의 가중치가 그래프의 시작점에 고정되어있을 때 모든 정점으로 가는 최단 경로의 거리 합을 모두 구하는 알고리즘
- 에드거 다익스트라가 1959년에 발표함

7-2. 다익스트라 로직

- 1) 출발 노드에서 가중치가 가장 작은 노드로 이동
-> 처음엔 한번에 갈 수 있는 경로로 이동
- 2) 1)에서 이동한 노드에서 이동한 노드에서 이동할 수 있는 노드들 가중치 False로 임시 저장해놓기
-> 나중에 계속 순회하면서 값이 작아지면 갱신해야하기 때문에 확정해놓으면 안됨
- 3) 2)에서 False로 임시 저장해놓은 노드들 중 가중치 합이 가장 작은 노드부터 이동
- 4) 3)에서 이동한 노드에서 갈 수 있는 노드 이동해서 가중치 False로 임시 저장
- 5) 1)~4) 과정 반복하다가 처음에 False로 저장해놓은 값보다 더 작은 가중치 합이 계산되면 새로 갱신
- 6) 존재하는 노드를 한번씩 다 둘러본 후에 임시저장되어있는 False 값(계속 갱신되어 결정된 최종 값)을 True로 확정

7-3. 다익스트라 알고리즘의 한계

- 이미 지나온 길은 이후에 다시는 쳐다보지않음
- 만약, 그 지나온 길을 거쳐서 가는 더 짧은 경로가 있다면 어떻게 할것인가?
- 따라서, 다익스트라 알고리즘은 모든 가중치가 양수인 경우에만 적용시킬 수 있는 최소 비용을 구하는 알고리즘

7-4. C++ 구현 방법

- 1) 시작 노드와 직접적으로 연결된 모든 정점들의 거리를 비교해서 업데이트 시켜주고, 시작 노드를 방문한 노드로 체크

```

1 for (int i = 1; i <= V; i++) Dist[i] = MAP[Start][i];
2 Dist[Start] = 0;
3 Select[Start] = true;

```

Colored by Cold

- Dist 배열 = 각 정점까지 가는데 드는 최소 비용 저장해놓은 배열
 - Select 배열 = 선택한 정점을 표시하기 위한 배열
 - Dist 배열의 초기값은 모두 INF / Select 배열의 초기값은 모두 false
- 2) 방문한 정점들과 연결되어있는 정점들 중 비용이 가장 적게 드는 정점 선택
 -> 해당 정점을 방문한 정점으로 선택

```

1 int Find_Shortest_Node()
2 {
3     int Min_Dist, Min_Idx;
4     Min_Dist = INF;
5     Min_Idx = -1;
6
7     for (int i = 1; i <= V; i++)
8     {
9         if (Select[i] == true) continue;
10        if (Dist[i] < Min_Dist)
11        {
12            Min_Dist = Dist[i];
13            Min_Idx = i;
14        }
15    }
16    return Min_Idx;
17 }

```

-
- 3) 2번 과정에 의해 갱신될 수 있는 정점들의 거리 갱신

```

1 void Update_Dist(int NewNode)
2 {
3     for (int i = 1; i <= V; i++)
4     {
5         if (Select[i] == true) continue;
6         if (Dist[i] > Dist[NewNode] + MAP[NewNode][i])
7         {
8             Dist[i] = Dist[NewNode] + MAP[NewNode][i];
9         }
10    }
11 }

```

Colored by Color St

4) 2~3번 과정 반복((정점의 수 -1)번 반복하게됨)

8. Greedy 알고리즘

8-1. 그리디 알고리즘 개요

8-1-1. 그리디 알고리즘이란?

- 매 순간 당장 눈앞에 보이는 최적의 상황만을 좇아 최종 해답에 도달하는 알고리즘 설계 기법

8-1-2. 그리디의 핵심 가정

- 탐욕적 선택 속성: 앞의 선택이 이후의 영향을 주지않음
- 최적 부분 구조: 문제에 대한 최종 해결 방법은 부분 문제에 대한 최적 문제 해결 방법으로 구성

8-2. 그리디 알고리즘 사례

8-3-1. 왜 다익스트라가 그리디인가

- 매 단계마다 방문하지 않은 노드 중 가장 비용이 적은 노드를 탐욕적으로 선택하기 때문
- 전체 경로의 최종 상태나 나중에 생길 변수 고려 X -> 지금 당장 가까운 노드를 고르는 현재 기준의 최적해 반복

8-3-2. 활동 선택 문제

- 서로 겹치지않는 최대 개수의 활동을 선택하는 문제

- 각 활동은 시작 시간과 종료 시간이 주어짐

9. 동적 계획법 알고리즘(DP)

9-1. 동적 계획법이란?

- 하나의 큰 문제를 여러 개의 작은 문제로 나눔 -> 그 결과를 저장
-> 다시 큰 문제를 해결할 때 사용

9-2. 적용시킬 수 있는 조건

- 중복되는 부분 문제: 기본적으로 문제를 나눔 -> 결과 값 재사용해서 전체 답 구함
-> 그래서 동일한 작은 문제들이 반복하여 나타나는 경우에 사용 가능
- 최적 부분 구조: 부분 문제의 최적 결과값 사용 -> 전체 문제의 최적 결과를 낼 수 있는 경우 사용 가능

9-3. 작동 과정

9-3-1. Bottom-up 방식

- 작은 문제부터 차례대로 해결 -> 전체 문제 해결
- 반복문 사용 -> 반복적으로 부분 문제 해결 -> 결과 배열에 저장

9-3-2. Top-Down 방식

- 큰 문제를 작은 부분 문제로 나누어 해결
- 재귀함수를 사용 -> 부분 문제들로 쪼갬 -> 중복 계산을 피하기 위해 이전에 계산한 값을 저장하는 Memoization 활용
- Memoization: 캐싱을 통해 이전 계산 결과를 저장하여 중복 계산을 피하는 것

10. Greedy 알고리즘 vs DP 알고리즘

10-1. 주요 차이점

10-1-1. 접근 방식

- DP: 큰 문제를 작은 하위 문제들로 나눔 -> 겹치는 부분의 결과를 재사용
- Greedy: 각 단계에서 당장 최선인 옵션을 고름 -> 이전 결정은 다시 보지않음

10-1-2. 최적해 보장

- DP: 모든 경우 탐색 -> 최적해 보장함
- Greedy: 최적해 보장하지 않음. 단, 조건이 성립하는 문제에서는 매우 빠르게 작동함

10-1-3. 계산 비용

- DP: 연산량 많음 -> 시간이 오래 걸림
- Greedy: 직관적이고 속도가 빠름